Complete React Frontend Development Guide

1. Introduction to React

React is a JavaScript library for building user interfaces, especially single-page applications.
It was developed by Facebook. React allows developers to create reusable UI components.
React follows a component-based architecture and uses a virtual DOM to improve performance.

2. Why Use React?

- Component-Based Architecture
- Reusable Code
- Fast Rendering with Virtual DOM
- Strong Community Support
- Easy Integration with APIs

3. Setting Up React

To create a React app:
1. Install Node.js
2. Use: npx create-react-app my-app
3. Navigate into folder: cd my-app
4. Start app: npm start
Alternative modern setup: Vite (npm create vite@latest)

4. Project Structure

- node_modules: Dependencies
- public: Static files
- src: Main source code
- App.js: Root component
- index.js: Entry point

5. JSX (JavaScript XML)

JSX allows writing HTML inside JavaScript.
Example:
const element = Hello World;
JSX must return a single parent element.
Use className instead of class.

6. Components in React

Two types:
1. Functional Components (Modern & Recommended)
2. Class Components (Older)

Example Functional Component:
function Header() {
return Hello;
}

7. Props

Props are inputs passed to components.
Example:

Access using: props.title
Props are read-only.

8. State and useState Hook

State allows components to store data.
useState is a React Hook.

Example:
const [count, setCount] = useState(0);
setCount(count + 1);

9. useEffect Hook

useEffect handles side effects like API calls.
Example:
useEffect(() => {
fetchData();
}, []);
Empty dependency array runs once.

10. Event Handling

React events use camelCase.
Example:
Click

11. Conditional Rendering

Render UI based on conditions.
Example:
{isLoggedIn ? : }

12. Lists and Keys

Render multiple elements using map().
Each element needs a unique key.
Example:
items.map(item => {item.name})

13. Forms in React

Controlled Components use state to handle input.
Example:
setName(e.target.value)} />

14. React Router

Used for navigation.
Install:
npm install react-router-dom
Use BrowserRouter, Routes, Route components.

15. API Integration

Use fetch or axios.
Example:
fetch('https://api.example.com/data')
.then(res => res.json())
.then(data => setData(data));

16. Styling in React

- CSS files
- Inline styles
- CSS Modules
- Tailwind CSS
- Styled Components

17. Performance Optimization

- React.memo()
- useMemo()
- useCallback()
- Lazy Loading (React.lazy)
- Code Splitting

18. Folder Structure Best Practices

- components/
- pages/
- hooks/
- services/
- assets/
- utils/

19. Deployment

Build project:
npm run build
Deploy to:
- Netlify
- Vercel
- Render
- GitHub Pages

20. Advanced Topics

- Context API
- Redux Toolkit
- Custom Hooks
- Higher Order Components
- Error Boundaries
- Suspense

Conclusion

React is one of the most powerful frontend libraries.
Mastering components, hooks, routing, and state management
will make you a strong frontend developer.
Practice by building real projects like:
- Todo App
- Weather App
- E-commerce UI
- Blog Website